optimize_anything: A Universal API for Optimizing Text Artifacts via LLM-Driven Search

Anonymous Author(s)

Abstract

A surprisingly wide range of optimization problems—from accelerating CUDA kernels to evolving agent architectures to refining cloud scheduling policies—can be formulated as optimizing a text artifact: if the artifact can be serialized to a string and its quality measured, an LLM can reason about it and propose improvements. We present `optimize_anything`, a declarative API that unifies three optimization paradigms under one interface: *single-task search* (solve one hard problem), *multi-task search* (solve a batch of related problems with cross-transfer), and *generalization* (build a skill that transfers to unseen problems). Two mechanisms underpin its effectiveness. First, *Actionable Side Information* (ASI)—diagnostic feedback such as compiler errors, profiler traces, or rendered images—is elevated to a first-class evaluator contract, enabling the LLM proposer to make targeted, diagnostic-driven improvements rather than blind mutations. Second, *Pareto-efficient search* across per-example or per-metric scores preserves complementary strengths across candidates rather than averaging them away. Across six domains, `optimize_anything` matches or outperforms domain-specific tools: it evolves agent architectures that nearly triple Gemini Flash's ARC-AGI accuracy (32.5% → 89.5%), discovers cloud scheduling algorithms that cut costs by 40%, generates CUDA kernels where 87% match or beat PyTorch baselines, and outperforms AlphaEvolve at circle packing. Ablations show that multi-task search outperforms independent single-task optimization on CUDA kernels via cross-problem transfer, and that ASI yields faster convergence and higher final scores than score-only feedback on prompt optimization.

CCS Concepts

• **Computing methodologies** → **Natural language processing**; *Neural networks*; **Artificial intelligence**.

Keywords

LLM optimization, text artifact optimization, evolutionary search, prompt engineering, agentic systems, Pareto optimization

ACM Reference Format:
Anonymous Author(s). 2026. `optimize_anything`: A Universal API for Optimizing Text Artifacts via LLM-Driven Search. In *Proceedings of ACM Conference on AI and Agentic Systems (ACM CAIS '26)*. ACM, New York, NY, USA, 9 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Recent work has demonstrated that large language models (LLMs) can serve as effective optimizers when paired with automated evaluation: FunSearch [18] discovers mathematical constructions, AlphaEvolve [14] designs algorithms for Google's infrastructure, and GEPA [1] sets the state-of-the-art for prompt optimization. These systems share a common insight—an LLM can read diagnostic feedback from a failed attempt, reason about why it failed, and propose a targeted fix—but each framework exposes different abstractions (island topologies, mutation templates, cascade evaluators) that couple the optimization logic to framework-specific mechanisms.

We observe that a surprisingly wide range of problems can be formulated as optimizing a text artifact. Whether the artifact is a CUDA kernel, a cloud scheduling policy, an agent architecture, or a system prompt, the structure is the same: serialize the artifact as a string, evaluate it, and let an LLM propose improvements based on diagnostic feedback. This observation suggests a much simpler interface is possible.

We present `optimize_anything`, a declarative API that implements this insight. The user provides a seed artifact (or a natural-language description of what is needed), an evaluator that returns a score and optional diagnostic feedback, and optionally a dataset. The system handles prompt construction, reflection, candidate selection, and search strategy. This declarative design, inspired by DSPy's [8] principle of *programming—not prompting*, means the same API call works whether one is optimizing a CUDA kernel or an agent architecture.

`optimize_anything` makes two key contributions beyond prior LLM-evolution frameworks:

(1) **Three unified optimization modes.** Prior frameworks (AlphaEvolve [14], OpenEvolve [12], ShinkaEvolve [13]) operate exclusively in single-task mode: optimize one artifact for one problem. `optimize_anything` additionally supports *multi-task search*, where solving a batch of related problems enables cross-transfer of optimization patterns, and *generalization*, where the optimized artifact must perform well on unseen examples. These modes enable optimization tasks that prior frameworks cannot directly express, such as discovering agent architectures from scratch (§4.3), learning prompts that generalize to unseen test sets (§4.4), and optimizing coding agent skills that transfer across models (§4.1).

ACM CAIS '26, San Jose, CA, USA
2026. ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

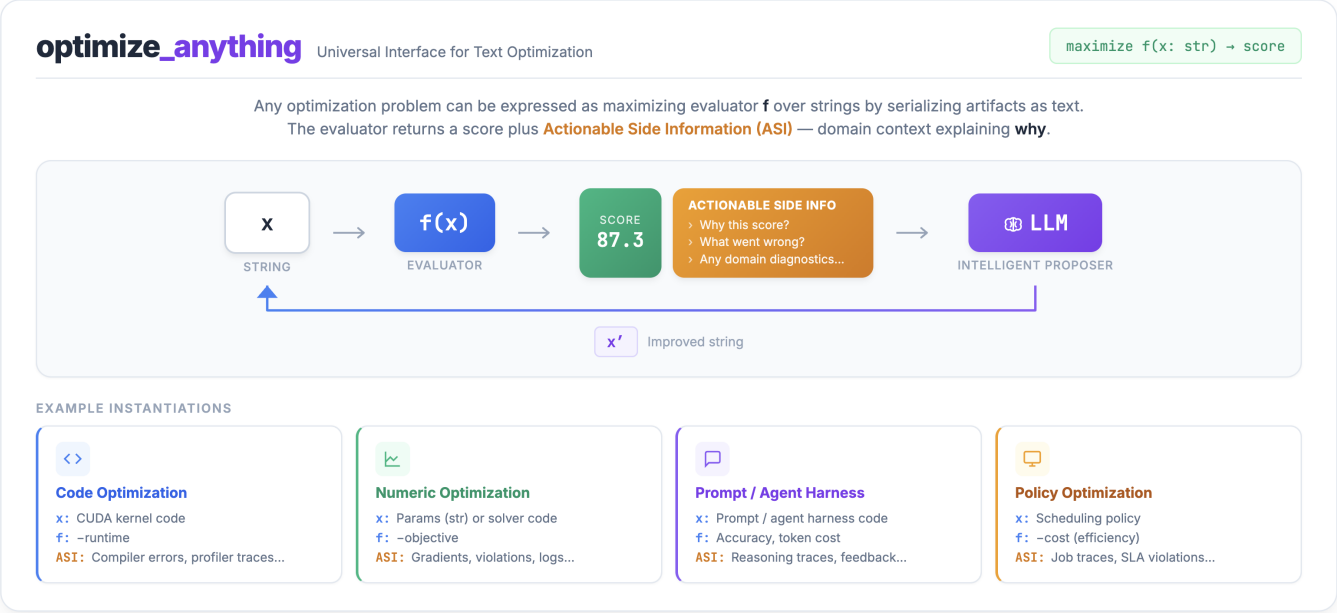


Figure 1: The `optimize_anything` loop: a text artifact x is passed to an evaluator $f(x)$ which returns a score plus diagnostic feedback (ASI), which is consumed by an LLM proposer to produce an improved artifact. The same API instantiates across domains: code optimization, prompt tuning, agent architecture search, and policy discovery.

(2) **Actionable Side Information (ASI) as a first-class API concept.** Prior frameworks support diagnostic feedback through ad-hoc mechanisms. `optimize_anything` makes it a uniform part of the evaluator contract: any diagnostic the evaluator can produce—stack traces, profiler data, rendered images, structured error reports—flows to the proposer through a single interface. We show empirically that ASI yields faster convergence and higher final scores than score-only feedback (§4.8).

These contributions are supported by Pareto-efficient search [1, 11]: candidates are selected based on per-example or per-metric Pareto dominance rather than aggregate scores, preserving complementary strengths across iterations. Table 2 provides a detailed comparison.

We evaluate `optimize_anything` across six domains spanning all three optimization modes (Table 1). Key results include: (i) evolved agent architectures nearly triple Gemini Flash's ARC-AGI accuracy (32.5% → 89.5%); (ii) discovered cloud scheduling algorithms cut costs by up to 40%, topping the ADRS leaderboard; (iii) 87% of generated CUDA kernels match or beat PyTorch baselines, with multi-task mode outperforming dedicated single-task optimization; (iv) prompt optimization improves GPT-4.1-mini's AIME accuracy from 46.67% to 60.00%; and (v) circle packing outperforms AlphaEvolve's published solution. Ablations confirm that both multi-task search and ASI contribute meaningful gains.

2 The `optimize_anything` API

2.1 Core Interface

At its simplest, `optimize_anything` requires a seed artifact and an evaluator:

```
import gepa.optimize_anything as oa

def evaluate(candidate: str) -> float:
    score, diagnostic = run_my_system(candidate)
    oa.log(f"Error: {diagnostic}") # captured as ASI
    return score

result = oa.optimize_anything(
    seed_candidate="<your artifact>",
    evaluator=evaluate,
)
```

The evaluator takes a candidate string and returns a score (higher is better). `oa.log()` routes output to the LLM proposer as Actionable Side Information (ASI)—diagnostic feedback the proposer reads during reflection. For richer diagnostics, evaluators return a structured dictionary alongside the score:

```
def evaluate(candidate: str) -> tuple[float, dict]:
    result = execute_code(candidate)
    return result.score, {
        "Error": result.stderr,
        "Output": result.stdout,
        "Runtime": f"{result.time_ms:.1f}ms",
    }
```

ASI can include open-ended text, structured data, multiple sub-scores, or images (via `gepa.Image`) for vision-capable LLMs.

The full signature is:

Table 1: Summary of experimental results across six domains. "Mode" indicates which optimization paradigm is used: S = single-task search, M = multi-task search, G = generalization. All results use `optimize_anything` with the indicated proposer LLM.

Domain	Mode	Proposer	Key Result
Agent Skills (§4.1)	G	Claude Opus 4.6	100% pass rate (+20.7pp)
Cloud Sched. (§4.2)	G	Gemini 3 Pro	40.2% cost savings
ARC-AGI (§4.3)	G	Gemini 3 Flash	89.5% test (+57pp)
AIME Prompts (§4.4)	G	GPT-5	60.0% test (+13.3pp)
CUDA Kernels (§4.5)	M	GPT-5	87% match baseline
Circle Pack. (§4.6)	S	Claude Opus 4.5	Beats AlphaEvolve

Table 2: Comparison of `optimize_anything` with prior LLM-evolution frameworks. Only `optimize_anything` supports multi-task search and generalization modes, and provides diagnostic feedback as a first-class API concept.

Feature	AlphaEv.	OpenEv.	ShinkaEv.	optimize_anything
Single-task search	✓	✓	✓	✓
Multi-task search				✓
Generalization				✓
ASI as API contract				✓
Image feedback				✓
Declarative interface				✓
No mutation templates				✓
Open-source		✓	✓	✓

```python
def optimize_anything(
    seed_candidate=None, # Starting artifact
    evaluator=..., # Score + ASI
    dataset=None, # Training examples
    valset=None, # Validation set
    objective=None, # Natural language goal
    background=None, # Domain knowledge
    config=None, # Engine settings
) -> GEPAResult:
```

Notice what is *absent*: no mutation prompts, no task-specific templates, no island configurations, no `EVOLVE-BLOCK` markers (all common in prior frameworks). The user declares the *what* (artifact, evaluator, domain knowledge); `optimize_anything` handles the *how*.

2.2 Three Optimization Modes

Which mode is active depends solely on whether `dataset` and `valset` are provided:

Single-Task Search. No dataset. The candidate *is* the solution; the evaluator scores it directly. This is the mode that AlphaEvolve and OpenEvolve operate in. Example: in circle packing (§4.6), the artifact is the packing algorithm and the evaluator returns the packing score plus geometric diagnostics.

Multi-Task Search. A `dataset` of related tasks is provided; insights from solving one help solve the others. Example: in CUDA kernel generation (§4.5), each task is a PyTorch operation to accelerate. Multi-task mode discovers optimization patterns that transfer across problems, converging faster and solving more problems than single-task runs (§4.7). No prior LLM-evolution framework supports this mode.

Generalization. Both `dataset` and `valset` are provided; the optimized artifact must perform well on unseen examples. This is the mode that GEPA's prompt optimization [1] operates in; `optimize_anything` generalizes the pattern to any text artifact. Example: in agent architecture discovery (§4.3), the artifact is the entire agent, and it must generalize to unseen ARC-AGI puzzles.

3 Method

`optimize_anything` is powered by GEPA [1] as the optimization backend. The system overview is shown in Figure 1. We describe the two mechanisms that underpin its effectiveness and contrast `optimize_anything` with prior frameworks.

3.1 Problem Formulation

We formalize the text optimization problem as follows. Let $\mathcal{X}$ denote the space of text artifacts (strings). An evaluator $f : \mathcal{X} \times \mathcal{E} \to \mathbb{R} \times \mathcal{I}$ maps an artifact $x \in \mathcal{X}$ and an optional example $e \in \mathcal{E}$ to a score $s \in \mathbb{R}$ and side information $\iota \in \mathcal{I}$. The three modes correspond to:

Single-task search: $\mathcal{E} = \emptyset$; maximize $f(x)$ directly. The artifact *is* the solution (e.g., a packing algorithm).

Multi-task search: Given a dataset $\mathcal{D} = \{e_1, \ldots, e_n\}$ of related problems, find an artifact x (e.g., a kernel-generation prompt) maximizing $\frac{1}{n}\sum_i f(x, e_i)$. Cross-transfer arises because the Pareto frontier preserves patterns that work across problems.

Generalization: Given training set $\mathcal{D}_{\text{train}}$ and validation set $\mathcal{D}_{\text{val}}$, find x maximizing performance on $\mathcal{D}_{\text{val}}$. Search uses $\mathcal{D}_{\text{train}}$; $\mathcal{D}_{\text{val}}$ measures generalization to unseen examples. This generalizes classical machine learning: the artifact may be a prompt, an agent, or a policy.

3.2 Actionable Side Information (ASI)

Classical optimization reduces all diagnostic context to a single scalar. The optimizer knows *that* a candidate failed, but not *why*—one cannot show a Bayesian optimizer a stack trace. LLM-evolution frameworks changed this by feeding execution results into LLM proposers, but the "evolutionary" framing suggests a blind process of mutate–evaluate–select. When an LLM reads a compiler error, diagnoses a logic bug, and proposes a targeted fix, that is an engineer iterating on a prototype.

`optimize_anything` leans into this by making diagnostic feedback a first-class part of the evaluator contract. The evaluator returns both a score and a `side_info` dictionary containing any diagnostic the evaluator can produce:

- **Text:** compiler errors, runtime exceptions, profiler summaries, natural-language critiques.
- **Structured data:** per-test-case results, sub-scores for multiple objectives, execution traces.
- **Images:** rendered SVGs, 3D model screenshots, or chart visualizations, enabling VLM proposers to *see* what they are improving.

ASI is the text-optimization analogue of the gradient. Where gradients tell a numerical optimizer which direction to move, ASI tells the LLM proposer *why* a candidate failed and *how* to fix it. During a dedicated reflection step, the proposer reasons over this signal to diagnose failures and propose targeted improvements.

Prior frameworks expose feedback through framework-specific mechanisms; ASI provides a uniform interface that makes it trivial to surface any diagnostic. The key design choice is that ASI is *opt-in but zero-friction*: evaluators that return only a score work fine, and existing `print()` statements can be captured automatically via `capture_stdio=True`.

3.3 Pareto-Efficient Search

Even when optimizing a single objective, evaluating candidates across multiple examples or metrics produces richer signal than a scalar aggregate. The naive approach collapses that signal into one average score and always selects the top candidate. This stalls fast: averaging hides which aspects are strong and which are weak, and the proposer tries to improve everything at once.

`optimize_anything` does two things differently. First, it tracks scores per task (from `dataset`) or per metric (from sub-scores in ASI) individually and maintains a **Pareto frontier**: any candidate that is the best at *something* survives, even if its average is suboptimal. Second, each reflection step shows the proposer a minibatch of just 2–3 examples instead of all of them, enabling focused, targeted improvements on that subset.

Over iterations, the frontier accumulates complementary strengths. Candidates that excel at different tasks are preserved and their strategies recombined. This mechanism also powers multi-task search: when optimizing across related problems, the frontier preserves candidates that excel on different tasks, and strategies discovered for one problem transfer to others (§4.7).

Candidate selection. Following GEPA [1], candidates are selected for mutation proportionally to their frequency on the Pareto front. Specifically, for each task i in the dataset, let $\mathcal{P}^*[i]$ denote the set of candidates achieving the highest score on i. Dominated candidates—those whose score vectors are component-wise $\leq$ some other candidate—are pruned.

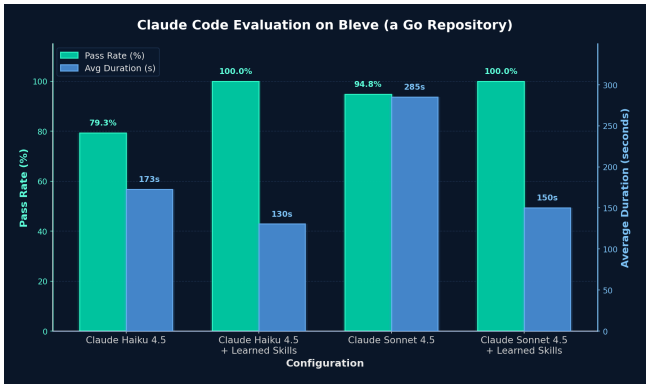


Figure 2: Claude Code on the Bleve repository. Optimized skills boost pass rates to near-perfect while reducing resolve time by 47%. Skills transfer across models without reoptimization.

The remaining candidates are sampled with probability proportional to $|\{i : \Phi \in \mathcal{P}^*[i]\}|$, focusing exploration on widely effective solutions.

Reflection and mutation. Given a selected candidate Φ_k and a minibatch $\mathcal{M}$ of examples, the system executes Φ_k on $\mathcal{M}$, collects scores and ASI, and presents them to the proposer LLM in a structured reflection prompt. The proposer diagnoses failures using the ASI and produces an updated artifact Φ'. If Φ' improves on the minibatch, it is fully evaluated and added to the candidate pool.

4 Experiments

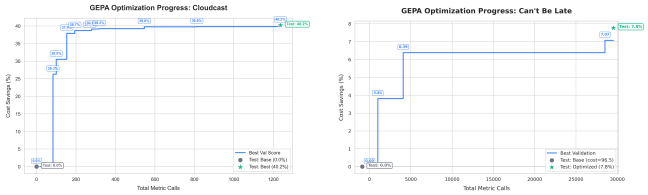
We evaluate `optimize_anything` across six domains spanning all three optimization modes. For each, we describe the artifact, evaluator, ASI design, and results. We then present ablation studies on multi-task search (§4.7) and ASI (§4.8).

4.1 Coding Agent Skills (Generalization)

Setup. Skills are natural-language instructions and best practices for working with a specific codebase. The evaluator runs a coding agent on repository tasks and scores whether it resolves them; the optimized skills must generalize to unseen tasks. We optimize skills for the Bleve search library and evaluate transfer to Claude Code with both Haiku 4.5 and Sonnet 4.5.

ASI design. The evaluator returns task descriptions, agent traces (tool calls, code edits, errors), test outcomes, and resolution time.

Results. Optimized skills boost Haiku 4.5's pass rate from 79.3% to 98.3% and Sonnet 4.5's from 94.8% to 100%, while cutting resolution time by 47% (Figure 2). Critically, skills discovered for one model transfer effectively to another without reoptimization, demonstrating the generalization mode's ability to learn model-agnostic repository knowledge.



(a) CloudCast: 40.2% cost savings. **(b) Can't Be Late: 7.8% savings.**

Figure 3: Optimization trajectories for cloud scheduling. Both use generalization mode with train/val splits over infrastructure scenarios.

4.2 Cloud Scheduling Algorithms (Generalization)

Setup. We optimize two cloud infrastructure algorithms from the ADRS benchmark. **CloudCast** discovers broadcast routing strategies for multi-cloud data transfer, minimizing egress cost. **Can't Be Late** learns scheduling policies deciding when to use cheap preemptible SPOT instances versus reliable ON_DEMAND instances to meet deadlines. Both use generalization mode with training/validation splits over infrastructure scenarios.

ASI design. For CloudCast: per-partition routing decisions, edge utilizations, cost breakdowns. For Can't Be Late: spot-availability patterns, instance-usage timelines, segment counts (SPOT vs ON_DEMAND vs restarts).

Results. CloudCast achieves 40.2% cost savings over Dijkstra routing (Figure 3a), evolving from a baseline shortest-path algorithm to a provider-aware Steiner tree approach that jointly optimizes for egress cost and transfer latency. Can't Be Late achieves 7.8% cost savings (Figure 3b), evolving a simple deadline-check heuristic into an adaptive strategy with state tracking for spot-unavailability patterns, break-even switching cost analysis, and graduated decision thresholds based on slack ratio. Both results top the ADRS leaderboard, outperforming OpenEvolve, ShinkaEvolve, and expert-designed heuristics. The evolved artifacts are qualitatively different from their seeds: CloudCast discovers provider-aware routing with Steiner tree relaxation (concepts not present in the Dijkstra seed), while Can't Be Late learns to track persistent spot unavailability and compute overhead-aware switching costs (behaviors absent from the greedy seed).

4.3 ARC-AGI Agent Architecture (Generalization)

Setup. Rather than optimizing a prompt, we optimize the *entire agent system*: code, sub-agent architecture, control flow, helper functions, and prompts are all treated as a single text artifact. The seed is a 10-line naive agent (one LLM call); GEPA evolves it into a 300+ line system. The artifact must generalize to unseen ARC-AGI [4] puzzles.

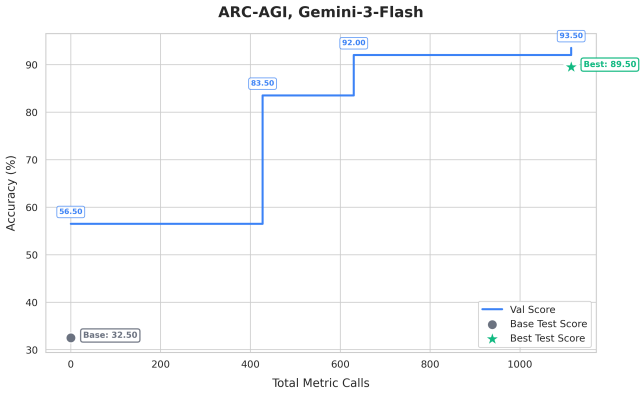


Figure 4: ARC-AGI agent architecture evolution with Gemini 3 Flash. Validation accuracy reaches 93.5%; test accuracy improves from 32.5% to 89.5%.

ASI design. Training/test grid examples, per-puzzle scores, internal model outputs, LLM costs, error tracebacks, and code execution results.

Results. Using Gemini 3 Flash as both the proposer and the underlying agent model, test accuracy improves from 32.5% to **89.5%**—a 57 percentage point gain (Figure 4). The evolved architecture implements a 4-stage pipeline: (1) rule induction via pattern analysis, (2) code generation with `exec()`-based verification, (3) iterative debugging with up to 2 fix attempts, and (4) structured fallback from code-first to direct LLM prediction. This represents a qualitative leap: the system discovers architectural patterns (verify-then-fallback, iterative refinement) that typically require significant manual engineering.

4.4 AIME Prompt Optimization (Generalization)

Setup. We optimize a system prompt for GPT-4.1-mini on AIME (American Invitational Mathematics Examination) competition problems. Training uses AIME 2022–2024; testing uses AIME 2025.

ASI design. The evaluator returns each problem statement, the model's reasoning chain, extracted answer, ground truth, and a correct/incorrect flag.

Results. Pure prompt optimization improves GPT-4.1-mini from 46.67% to **60.00%** on AIME 2025 (Figure 5), a 13.3 percentage point gain from changing only the system prompt. The optimized prompt is a structured reasoning framework with 6 explicit rules: (1) restate the problem formally, (2) set up notation before computing, (3) verify each logical step, (4) recognize and abandon dead ends, (5) stay focused on the specific question asked, and (6) isolate the final answer clearly. The seed prompt was a single generic sentence ("Solve the math problem carefully"). This result demonstrates that generalization mode can discover effective reasoning strategies from minimal seeds, consistent with GEPA's state-of-the-art prompt optimization results [1].

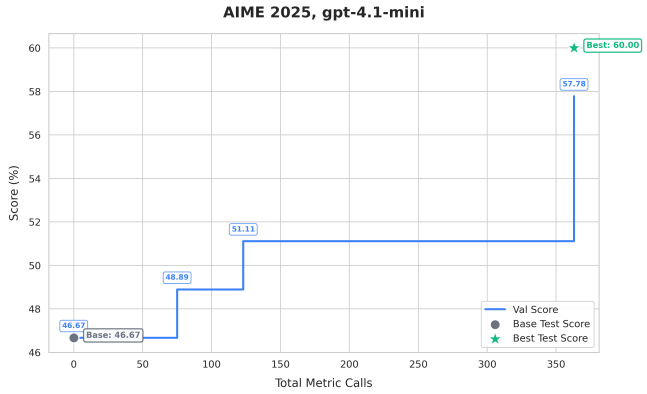


Figure 5: AIME prompt optimization for GPT-4.1-mini. Validation score improves from 46.67% to 57.78%; test score reaches 60.00%.

4.5 CUDA Kernel Generation (Multi-Task Search)

Setup. We generate CUDA kernels for 31 reference PyTorch operations from KernelBench [16], evaluated on a V100 32GB GPU. The 31 problems span diverse operations: matrix multiplications, convolutions, reductions, element-wise ops, and normalization layers. Under the hood, GEPA evolves the prompt that drives kernel generation; in multi-task mode, insights discovered for one problem (e.g., how to handle memory coalescing) transfer to others automatically through the shared Pareto frontier.

ASI design. The evaluator compiles the generated kernel, runs correctness tests (max absolute error vs. PyTorch reference), and benchmarks wall-clock time. ASI includes: (i) NVCC compiler errors with line numbers, (ii) correctness test failures with actual vs. expected outputs, (iii) relevant CUDA documentation snippets, and (iv) speedup ratio vs. the PyTorch baseline.

Results. 87% of generated kernels match or beat the PyTorch baseline performance; 48% achieve 10%+ speedups, and 25% achieve 20%+ speedups (Figure 6). The evolved kernels employ techniques such as float4 vectorization, two-pass algorithms (compute statistics, then normalize), warp shuffle reductions, and shared memory tiling. Multi-task mode's advantages are analyzed in §4.7.

4.6 Circle Packing (Single-Task Search)

Setup. Pack n=26 circles to maximize the sum of radii within a unit square. The artifact is the packing algorithm code; the evaluator returns the score plus geometric diagnostics.

ASI design. Circle positions, radii, constraint violations, overlap distances, boundary violations, and a rendered visualization of the packing.

Results. GEPA reaches a score of 2.63598+, outperforming AlphaEvolve, OpenEvolve, and ShinkaEvolve (Figure 7). The evolved algorithm is a bilevel optimizer: an LP over radii

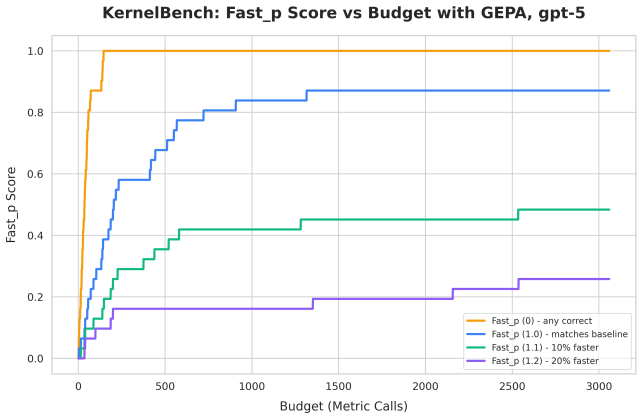


Figure 6: KernelBench results (GPT-5 as proposer). Fast$_p(s)$: fraction of kernels achieving speedup $\geq s$. 87% match baseline; 25% are 20%+ faster.

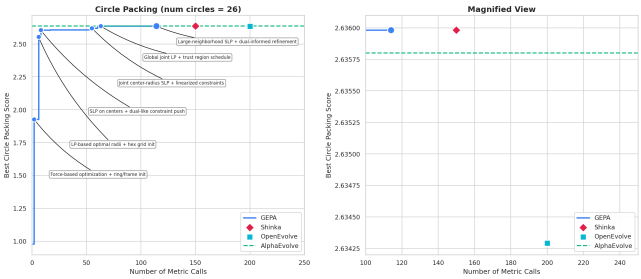


Figure 7: Circle packing (n=26). Gepa outperforms AlphaEvolve, ShinkaEvolve, and OpenEvolve, reaching a higher score with fewer evaluations.

with dual-variable gradients for L-BFGS-B center optimization, augmented by CMA-ES exploration and diverse seeding strategies.

4.7 Ablation: Multi-Task vs. Single-Task Search

To quantify the benefit of cross-task transfer, we take the 10 KernelBench problems where multi-task mode performed best and re-optimize each from scratch in single-task mode with an equivalent per-problem budget.

Figure 8 shows that multi-task mode consistently outperforms single-task search across all speedup thresholds. At the Fast$_p$(1.0) threshold (matching baseline performance), multi-task mode solves problems faster and reaches a higher fraction solved. At Fast$_p$(1.2) (20% speedup), the gap widens: single-task mode plateaus early, while multi-task continues improving.

The mechanism is cross-transfer via the Pareto frontier: optimization patterns discovered for one kernel (e.g., vectorized memory access, warp-level reductions) are preserved on the frontier and inform proposals for other kernels. In

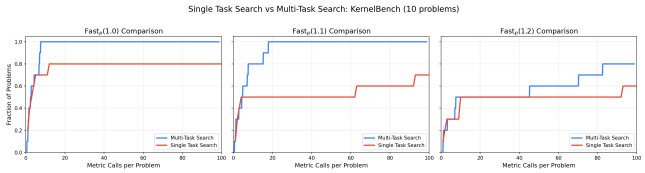


Figure 8: Single-task vs. multi-task mode on 10 selected KernelBench problems. Multi-task (solid) consistently outperforms single-task (dashed) at all speedup thresholds, converging faster and solving more problems.

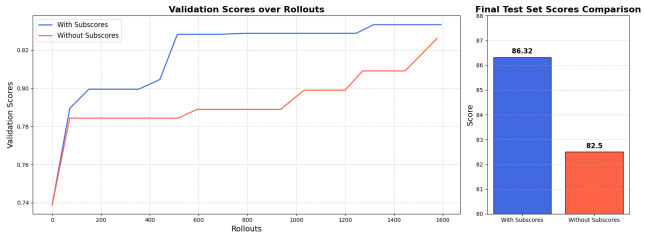


Figure 9: Ablation: prompt optimization with vs. without ASI sub-scores on the Facility Support Analysis dataset. ASI accelerates convergence (left) and improves final test performance (right): 86.32 vs. 82.5.

single-task mode, each problem must independently discover these patterns.

We note that cross-transfer is not universally beneficial. On circle packing, where all 26 circles share the same geometric constraints and there is only a single problem, multi-task mode offers no advantage over single-task search. The benefit emerges when problems are related but distinct, with transferable optimization strategies.

4.8 Ablation: Actionable Side Information

To isolate the contribution of ASI, we compare `optimize_anything` with and without structured diagnostic feedback (sub-scores) on prompt optimization for the Facility Support Analysis dataset. In the "with ASI" condition, the evaluator returns per-aspect sub-scores alongside the aggregate score. In the "without ASI" condition, only the aggregate score is returned.

Figure 9 shows two effects. First, ASI accelerates convergence: the "with sub-scores" condition reaches a validation score of 0.80 within 100 rollouts, while the score-only condition requires approximately 600 rollouts to reach the same level. Second, ASI improves final performance: the test score with sub-scores is 86.32 versus 82.5 without—a 3.82-point gap.

The explanation is that sub-scores provide per-aspect signal: the proposer can identify that a candidate excels at aspect A but fails at aspect B, and target its revision accordingly. Without sub-scores, the proposer receives only "the overall score improved slightly," losing the diagnostic signal that enables targeted improvements. This is consistent with the broader insight that ASI converts blind search into diagnostic-driven engineering.

5 Related Work

LLM-based program evolution. AlphaEvolve [14] pioneered the LLM-evolution paradigm, using Gemini models with island-based MAP-Elites [11] to discover algorithms for Google's infrastructure. OpenEvolve [12] provides an open-source reimplementation with model-agnostic support. ShinkaEvolve [13] extends the paradigm with diff-based mutations for sample efficiency and behavioral descriptors for diversity. FunSearch [18] applies evolutionary LLM search to mathematical discovery. EvoPrompting [3] evolves code for neural architecture search. All operate exclusively in single-task mode and expose framework-specific abstractions (island topologies, prompt samplers, evolve-block markers). `optimize_anything` strips the interface to its declarative essence, adds multi-task and generalization modes, and elevates diagnostic feedback to a first-class API concept.

Prompt optimization. GEPA [1] combines reflective mutation with Pareto-efficient search for prompt optimization, outperforming both MIPROv2 [15] and GRPO [19]. `optimize_anything` builds on GEPA and extends it beyond prompts to arbitrary text artifacts. Other prompt optimization methods include OPRO [21], APE [24], ProTeGi [17], and PromptBreeder [6]. TextGrad [22] uses LLM-generated "gradients" for text optimization.

LLM self-improvement and reflection. Reflexion [20] uses verbal reinforcement for agent self-correction. Self-Refine [10] applies iterative self-feedback. Evolution through Large Models [9] explores LLMs as mutation operators. `optimize_anything`'s ASI mechanism generalizes these ideas by making diagnostic feedback a declarative evaluator contract rather than a hardcoded self-critique.

Agent architecture search. ADAS [7] and AFlow [23] search over agent architectures. `optimize_anything`'s generalization mode subsumes these as special cases: the artifact is the agent code, the evaluator runs it on tasks, and the system evolves both architecture and prompts jointly.

6 Discussion

When does multi-task search help? Our experiments reveal that cross-task transfer is most beneficial when problems share underlying optimization patterns but differ in their specifics. CUDA kernel generation exemplifies this: memory coalescing, vectorized access patterns, and warp-level reductions are strategies that apply across operations but manifest differently for each kernel. Multi-task mode discovers these patterns once and transfers them, while single-task mode must rediscover them independently for each problem. Conversely, circle packing—a single problem where all circles share identical constraints—does not benefit from multi-task mode because there are no "related problems" to transfer

from. We hypothesize that multi-task search will be most effective for code generation tasks where common optimization idioms exist across a family of programs.

The role of ASI across domains. The ASI ablation on prompt optimization demonstrates the value of structured diagnostic feedback, but the mechanism operates differently across domains. For code optimization (CUDA, circle packing), ASI primarily surfaces compiler errors and runtime diagnostics that pinpoint exact failure locations. For agent architecture evolution (ARC-AGI), ASI provides per-puzzle execution traces that reveal which architectural components are failing. For cloud scheduling, ASI exposes the temporal structure of decisions—when the policy chose SPOT vs. ON_DEMAND and whether that choice was optimal in retrospect. In each case, ASI converts a scalar signal into actionable diagnostic information that enables the proposer to make targeted rather than random improvements.

Artifacts optimized by `optimize_anything`. A notable property of the results is the diversity of artifacts optimized through a single interface. The optimized artifacts range from 6-rule structured prompts (AIME) to 300+ line agent architectures (ARC-AGI) to 900+ line bilevel optimization algorithms (circle packing). The system discovers qualitatively different solution strategies: multi-stage pipelines with fallback logic (ARC-AGI), provider-aware Steiner tree algorithms (CloudCast), and break-even cost analysis with state tracking (Can't Be Late). These are not minor refinements of the seed—they represent novel algorithmic ideas generated through the interaction of LLM reasoning and diagnostic feedback.

7 Limitations

`optimize_anything` inherits limitations from LLM-based optimization. First, the quality of proposals depends on the proposer LLM's capabilities; weaker models produce weaker candidates. Second, evaluation cost can be high when the evaluator involves expensive operations (e.g., running a full agent on many tasks). Third, the system assumes the artifact is representable as text; optimization of continuous parameters or binary artifacts requires a text-based proxy. Fourth, while multi-task search provides cross-transfer benefits on related problems, the degree of benefit depends on how related the problems are—we showed circle packing does not benefit from multi-task mode. Finally, our ASI ablation is conducted on a single prompt optimization task; broader ablations across domains would strengthen the finding.

8 Conclusion

`optimize_anything` demonstrates that a simple declarative interface—seed artifact, evaluator, optional dataset—is sufficient to optimize text artifacts across a wide range of domains, matching or outperforming purpose-built tools. The key ideas are (1) three unified optimization modes under one API, (2) Actionable Side Information as a first-class evaluator contract, and (3) Pareto-efficient search across metrics and examples. As LLMs become more capable, we expect the range of problems expressible as text optimization to expand further. The `optimize_anything` API is designed to be backend-agnostic: as new optimization strategies emerge, they can be plugged in without changing user code.

References

[1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning. In *International Conference on Learning Representations (ICLR)*.

[2] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. 2019. Optuna: A Next-generation Hyperparameter Optimization Framework. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.

[3] Angelica Chen, David Dohan, and David So. 2023. EvoPrompting: Language Models for Code-Level Neural Architecture Search. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[4] François Chollet. 2019. On the Measure of Intelligence. *arXiv preprint arXiv:1911.01547* (2019).

[5] Ian Dewancker, Michael McCourt, and Scott Clark. 2018. SigOpt EvalSet: A Multi-Task Benchmark for Blackbox Optimization. *NeurIPS Workshop on Bayesian Optimization* (2018).

[6] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2024. PromptBreeder: Self-Referential Self-Improvement via Prompt Evolution. *arXiv preprint arXiv:2309.16797* (2024).

[7] Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated Design of Agentic Systems. In *arXiv preprint arXiv:2408.08435*.

[8] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Mober, Prem Shah, et al. 2024. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *International Conference on Learning Representations (ICLR)*.

[9] Joel Lehman, Jonathan Gordon, Shyamal Jain, Kamal Ndousse, Cathy Yeh, and Kenneth O. Stanley. 2022. Evolution through Large Models. *arXiv preprint arXiv:2206.08896* (2022).

[10] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).

[11] Jean-Baptiste Mouret and Jeff Clune. 2015. Illuminating search spaces by mapping elites. In *arXiv preprint arXiv:1504.04909*.

[12] Tung Doan Nguyen and Khanh Doan Nguyen. 2025. OpenEvolve: Open-Source AlphaEvolve. *arXiv preprint* (2025).

[13] Kei Nishimura-Gasparian, Matthew Ho, Alexander Ly, and Yifan Hao. 2025. ShinkaEvolve: Towards Open-Ended and Sample-Efficient Program Evolution. *Sakana AI Technical Report* (2025).

[14] Alexander Novikov, Ngoc-Hieu Nguyen, Lasse Espeholt, Yusuf Hamied, et al. 2025. AlphaEvolve: A Coding Agent for Scientific and Algorithmic Discovery. *Google DeepMind Technical Report* (2025).

[15] Krista Opsahl-Ong, Michael J Ryan, Josh Hardy, Jeremy Peng, and Omar Khattab. 2024. Optimizing Instructions and Demonstrations for Multi-Stage Language Model Programs. In *Empirical Methods in Natural Language Processing (EMNLP)*.

[16] Anne Ouyang, Simon Zheng, Kunhao Xia, and Ion Stoica. 2025. KernelBench: Can LLMs Write Efficient GPU Kernels? *arXiv preprint arXiv:2502.10517* (2025).

[17] Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic Prompt Optimization with "Gradient Descent" and Beam Search. In *Empirical Methods in Natural Language Processing (EMNLP)*.

[18] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Mateja Balog, M. Pawan Kumar, Emilien Dupont, Francisco J.R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. 2024.

Mathematical discoveries from program search with large language models. *Nature* 625 (2024), 468–475.
[19] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y.K. Li, Y. Wu, and Daya Guo. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. In *arXiv preprint arXiv:2402.03300*.
[20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems (NeurIPS)* (2023).
[21] Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large Language Models as Optimizers. In *International Conference on Learning Representations (ICLR)*.
[22] Mert Yüksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. 2024. TextGrad: Automatic "Differentiation" via Text. *arXiv preprint arXiv:2406.07496* (2024).
[23] Jiayi Zhang, Jinyu Guo, Yanbo Lin, Yibo Wan, Xiangkun Guo, Jian Zheng, Peiyi He, et al. 2024. AFlow: Automating Agentic Workflow Generation. *arXiv preprint arXiv:2410.10762* (2024).
[24] Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large Language Models Are Human-Level Prompt Engineers. In *International Conference on Learning Representations (ICLR)*.

A Blackbox Mathematical Optimization

We additionally evaluate `optimize_anything` in single-task search mode on blackbox mathematical optimization, using the 56-problem EvalSet benchmark [5] against Optuna [2]. Rather than tuning parameters within a fixed algorithm, `optimize_anything` optimizes the solver code itself, discovering bespoke algorithms for each problem.

With a budget of 8,000 evaluations per problem, `optimize_anything` ties Optuna on 40 problems, wins 7, and loses 9. On 10 selected problems where Optuna struggles with lower budgets (2,000 evaluations), `optimize_anything` finds better solutions on 7 out of 10. The mechanism: Optuna's fixed TPE-CMA-ES pipeline fails in predictable, structural ways (e.g., TPE's per-dimension sampling converges to trap basins; CMA-ES assumes smooth unimodal landscapes). `optimize_anything` tailors the solver to each problem—discovering L-BFGS-B for boundary optima and multi-start search for deceptive traps.

B Seedless Mode: 3D Unicorn

Every main experiment starts from a seed artifact. Seedless mode (`seed_candidate=None`) instead provides only a natural-language objective and lets the LLM bootstrap the first candidate. We demonstrate this on a 3D modeling task: generating a Python script (build123d + pyrender) that produces a 3D unicorn. The evaluator renders multi-view PNGs and asks a VLM to score them, passing images back as ASI. Starting from no code, `optimize_anything` iteratively refines geometry, proportions, and anatomical detail, producing a recognizable 3D unicorn that improves substantially over the zero-shot baseline.

C Detailed Algorithm

Algorithm 1 presents the core loop. For single-task search, the "dataset" is a singleton and per-example tracking reduces to per-metric tracking. For multi-task search, each dataset element is an independent problem. For generalization, scores on $\mathcal{D}$ guide search while a held-out `valset` measures generalization. The ParetoSelect subroutine follows GEPA [1]: it identifies non-dominated candidates and samples proportionally to their frontier frequency.

Algorithm 1 `optimize_anything`: Core optimization loop

Require: Artifact Φ_0, evaluator f, dataset $\mathcal{D}$, budget B
Require: Minibatch size b, Pareto set size n
Initialize candidates $\mathcal{P} \leftarrow [\Phi_0]$
Evaluate Φ_0 on $\mathcal{D}$; record per-example scores S
while budget B not exhausted **do**
  $k \leftarrow \textsc{ParetoSelect}(\mathcal{P}, S)$ ▷ Select based on frontier
  $\mathcal{M} \leftarrow$ minibatch of size b from $\mathcal{D}$
  Execute Φ_k on $\mathcal{M}$; collect scores and ASI
  $\Phi' \leftarrow \textsc{Reflect}(\Phi_k, \text{scores, ASI})$ ▷ LLM proposes fix
  if Φ' improves on $\mathcal{M}$ **then**
    Evaluate Φ' on full $\mathcal{D}$
    $\mathcal{P} \leftarrow \mathcal{P} \cup \{\Phi'\}$
    Update S; prune dominated candidates
  end if
end while
return $\Phi^* \in \mathcal{P}$ maximizing average score